COMPLETE REFERENCE · CHECKED AGAINST VERSION 2.1.310

Claude Code
Command Reference

Every terminal command, every option and every slash command inside a session — with an explanation of what it does.

Compiled by Christian's virtual server · 24 October 2026 · Update of the edition of 23 October 2026 (2.1.308)

How this list came about

Terminal commands and options (parts A and B) are read straight out of the installed version 2.1.310 — `claude --help` and the help of every subcommand.

The slash commands (part C) come from the command definitions inside the program itself. The extraction also picks up commands whose description is only assembled at runtime or fetched from a variable, and since this edition equally those whose *name* sits in a variable instead of being spelled out in the code. Both are artefacts of the code being minified at build time and say nothing about the command itself.

Commands that exist in the program but are switched off by a flag are listed separately at the end, so that nothing appears available that is not.

What has changed since the last edition

Compared with 2.1.308: **`/artifact-pr-review` has been removed**, the command that created a pull request review as a shareable artifact page. It has been gone since 2.1.309 without replacement and is no longer in the program. The changelog does not mention it.

New: the `--trust-workspace` option for `claude ultrareview`. It trusts the current folder for this one run without asking (for scripts and CI). Use it only in folders you trust. Otherwise there are no new commands and no changed descriptions; `claude --help` is character-for-character identical.

Correction: versions 2.1.307 and 2.1.308 listed `/shell_expansion` as a command that could not be read out. That was wrong. It is not a slash command but an internal check type of the shell command safety check, and is no longer reported.

Part A · Terminal commands and options

Call form: `claude [options] [command] [input]`. With no arguments it starts a session in the current folder.

Starting a session

`claude`	Starts a session in the current folder.
`claude "your question"`	Starts with a first message.
`claude -p "…"`	Answers once and exits — for scripts.

Options

`--add-dir <directories...>`	Additional folders Claude is allowed to touch.
`--agent <agent>`	Agent for this session; overrides the setting.
`--agents <json-or-file>`	Define custom agents as JSON, or with --print as the path to a file.

`--allow-dangerously-skip-permissions`	Allows turning off all prompts later, without doing so right away.
`--allowedTools, --allowed-tools <tools...>`	Allow only these tools, e.g. "Bash(git *) Edit".
`--append-system-prompt <prompt>`	Append text to the system prompt.
`--autocompact <auto|tokens>`	When to compact automatically (auto, or 100k–1M).
`--ax-screen-reader`	Screen-reader output: flat text, no borders, no animations.
`--bg, --background`	Start the session in the background and return right away. Prints the id that claude attach, logs, stop and rm take; claude agents lists them. With --resume , continues that session in the background under the same ID, or starts a copy and says so when the session is already running.
`--bare`	Frugal mode: no hooks, plugins, auto-memory or CLAUDE.md discovery.
`--betas <betas...>`	Send beta identifiers to the API (API key users only).
`--brief`	Lets Claude send you short progress notes while it works.
`--chrome`	Turn on the Claude-in-Chrome integration.
`--cloud [description|session_id|url]`	Create a cloud session or attach to an existing one.
`-c, --continue`	Continue the most recent conversation in this folder.
`--dangerously-skip-permissions`	Turn off all prompts. Only for sealed-off environments without internet.
`-d, --debug [filter]`	Turn on debugging, optionally filtered (e.g. --debug=hooks,mcp or --debug='!hooks,!mcp').
`--debug-file <path>`	Write the debug log to this file.
`--desktop`	Open the session in the Claude Desktop app instead of the terminal (pick it with --continue or --resume).
`--disable-slash-commands`	Turn off all skills and commands.
`--disallowedTools, --disallowed-tools <tools...>`	Block these tools.
`--effort <level>`	Thinking effort for the session: low, medium, high, xhigh, max.
`--environment <environment_id>`	Start a cloud session in a self-hosted environment.
`--exclude-dynamic-system-prompt-sections`	Keep machine-specific details out of the system prompt (better caching).
`--fallback-model <model>`	Backup model for when the chosen one is overloaded (now also outside --print).
`--file <specs...>`	Download files at startup (file_id:target_path).
`--fork-session`	On resume, issue a new session ID instead of continuing the old one.
`--forward-subagent-text`	Pass through the text and thinking of subagents as well.
`--from-pr [value]`	Resume the session belonging to a pull request (number or URL).
`-h, --help`	Show help.
`--ide`	Connect to the development environment automatically at startup.
`--include-hook-events`	Write hook events into the output stream as well (only with --print).
`--include-partial-messages`	Emit parts of the answer as they arrive (only with --print).

`--input-format <format>`	Input format: text or stream-json (only with --print).
`--json-schema <schema>`	JSON schema the answer is validated against (only with --print).
`--max-budget-usd <amount>`	Dollar ceiling for the API calls (only with --print).
`--mcp-config <configs...>`	Load MCP servers from JSON files or strings.
`--model <model>`	Model for this session.
`-n, --name <name>`	Display name for the session.
`--no-chrome`	Turn off the Chrome integration.
`--no-session-persistence`	Do not write the session to disk.
`--output-format <format>`	Output format: text, json or stream-json (only with --print).
`--permission-mode <mode>`	Permission mode: acceptEdits, plan, bypassPermissions, default.
`--permission-prompts <target>`	Who answers permission prompts with --print: "host" (the SDK host or --permission-prompt-tool) or "none" (auto-deny; the permission mode still decides everything else). Default: host.
`--plugin-dir <path>`	Load a plugin from a folder or archive, for this session only; a folder holding several plugins loads all of them.
`--plugin-url <url>`	Load a plugin archive from a URL, for this session only.
`-p, --print`	Print the answer and exit — for scripts and pipes.
`--prompt-suggestions [value]`	Turn on prompt suggestions.
`--remote-control [name]`	Start the session with Remote Control (phone, claude.ai).
`--remote-control-session-name-prefix <prefix>`	Name prefix for auto-named Remote Control sessions.
`--replay-user-messages`	Echo incoming user messages back for acknowledgement.
`--restricted`	Restricted mode: no tools that run commands or code, no WebFetch, no user, project or local settings files; file access only in the working directories.
`-r, --resume [value]`	Resume a conversation by ID or name, or open the picker.
`--safe-mode`	Start without any customisation: no CLAUDE.md, skills, plugins, hooks.
`--session-id <uuid>`	Use a fixed session ID.
`--setting-sources <sources>`	Which setting sources to load: user, project, local.
`--settings <file-or-json>`	Additional settings from a file or a JSON string.
`--strict-mcp-config`	Use only the MCP servers given via --mcp-config.
`--system-prompt <prompt>`	Replace the system prompt for the session.
`--system-prompt-snapshot <on|off>`	Record the system prompt once per conversation and reuse it verbatim on every request and resume (recommended: on). On by default (except under --bare outside cloud and background sessions); a --system-prompt or --append-system-prompt is recorded too, even if a later launch passes different text. off: render fresh on every request, for iterating on prompt text.
`--teleport [session]`	Resume a teleport session, optionally by session ID.
`--tmux`	Create a tmux session for the worktree (needs --worktree).
`--tools <tools...>`	Pick the built-in tools: "" for none, "default" for the default set, or a list.

`--verbose`	Override the verbose setting from the configuration.
`-v, --version`	Show the version number.
`-w, --worktree [name]`	Create a new git worktree for this session (name optional).

Part B · Subcommands

Calls of the form `claude <subcommand>`. Indented below each one, its own subcommands.

`claude agents`	Manage background agents.
`claude attach`	Open a background session in this terminal.
`claude auth`	Manage sign-in.
`claude auth login`	Sign in to the Anthropic account.
`claude auth logout`	Sign out.
`claude auth status`	Show sign-in status.
`claude auto-mode`	View or reset the settings of automatic mode.
`claude auto-mode config`	Print the effective configuration as JSON.
`claude auto-mode critique`	Get feedback on your own rules.
`claude auto-mode defaults`	Print the built-in default rules.
`claude auto-mode reset`	Reset to the default rules.
`claude doctor`	Check the installation. For the full examination use /doctor in a session.
`claude gateway`	Run a company gateway for sign-in and telemetry.
`claude import`	Take over the configuration of another AI tool.
`claude install`	Install the native build (stable, latest or a fixed version).
`claude logs`	Show a background session's recent terminal output.
`claude mcp`	Set up and manage MCP servers.
`claude mcp add`	Add an MCP server (stdio, http or sse).
`claude mcp add-from-claude-desktop`	Import servers from Claude Desktop (Mac and WSL only).
`claude mcp add-json`	Create a server from a JSON string.
`claude mcp get`	Show the details of one server.
`claude mcp list`	List the configured servers.
`claude mcp login`	Sign in to a server.
`claude mcp logout`	Sign out of a server.
`claude mcp remove`	Remove a server.
`claude mcp reset-project-choices`	Reset the per-project approvals.

`claude mcp serve`	Run Claude Code itself as an MCP server.
`claude plugin`	Manage plugins.
`claude plugin configure`	Show a plugin's options, or save values from stdin.
`claude plugin details`	Contents of a plugin and its estimated context cost.
`claude plugin disable`	Turn a plugin off.
`claude plugin enable`	Turn a plugin on.
`claude plugin eval`	Run test cases against a plugin and score them.
`claude plugin init|new`	Scaffold a new plugin.
`claude plugin install|i`	Install a plugin from a marketplace.
`claude plugin list`	List the installed plugins.
`claude plugin marketplace`	Manage marketplaces.
`claude plugin prune|autoremove`	Remove plugins that are no longer needed.
`claude plugin tag`	Create a git tag for a plugin release, checking the manifests.
`claude plugin test`	Run a mod's tests.
`claude plugin uninstall|remove`	Uninstall a plugin.
`claude plugin update`	Update a plugin.
`claude plugin validate`	Check a plugin for errors.
`claude purge`	Delete all stored state of a project (transcripts, tasks, file history, config entry).
`claude respawn`	Restart background sessions so they run the current version.
`claude rm`	Delete a background session, and its worktree when safe.
`claude setup-token`	Create a long-lived authentication token (for scripts and CI).
`claude stop|kill`	Stop a background session; its conversation is kept.
`claude ultrareview`	Start a multi-stage code review in the cloud.
`claude update|upgrade`	Check for updates and install them; package-manager installs update through the package manager.

Part C · Slash commands inside a session

133 available commands, in alphabetical order. In grey behind them: second names and expected arguments.

`/add-dir` <path>	Add another working folder.
`/advisor`	Let a stronger model weigh in at the important moments.
`/agents`	Removed. Subagents now live in .claude/agents/.
`/artifact-capabilities`	What published pages can do at runtime.
`/artifact-components`	A kit of reusable building blocks for published pages.

`/artifact-design`	Design rules for published pages.
`/artifact-diagramming`	Diagrams for published pages.
`/artifacts`	Browse published and shared artifacts.
`/auto-mode-setup` [--request-id <uuid>] (--wizard posture=\u2026 scope=\u2026 depth=\u2026 --propose | --expect-sha256 <64-hex> --apply-file <path>)	Set up and tune automatic mode.
`/autocompact` [auto|<tokens>]	Decide at which fill level the context is compacted automatically.
`/autofix-pr`	Watch the current pull request and fix what breaks.
`/background` also /bg · [prompt]	Send the session to the background and free up the terminal.
`/batch` <instruction>	Plan a large change, then have it carried out in parallel.
`/branch` [name]	Fork the conversation at this point.
`/brief`	Toggle brief mode.
`/btw` [question]	Ask a side question without disturbing the current thread.
`/bug` also /share · [report]	Report a bug or share the conversation.
`/chrome`	Open the settings for Claude in Chrome.
`/claim-credit`	Open the page of the offer shown at start-up.
`/claude-api`	Reference for the Claude API (models, prices, tools).
`/claude-code-docs` [question]	Answer questions about Claude Code itself (commands, flags, settings, hooks, skills), checked against the installed build.
`/claude-in-chrome`	Drive the browser: click, type, read pages.
`/clear` also /reset, /new · [name]	New session with an empty context; the old one is kept.
`/cloud-plugins`	Decide whether cloud sessions use the plugins enabled here.
`/code-review` also /review · […] [--fix] [--comment] [<pr#>|<branch>|<path>] [--max-findings <n>|all]	Review changes for bugs and simplifications.
`/color`	Colour of the prompt box for this session.
`/commit` [guidance]	Creates a git commit, with a fitting message written for it.
`/commit-push-pr`	Commit, push and open a pull request — in one go.
`/compact` <optional custom summarization instructions>	Free up context by summarising what happened so far.
`/config` also /settings · [key=value]	Open the settings.
`/context` [all]	Show context usage as a coloured grid.
`/copy`	Copy the last answer to the clipboard (/copy N for older ones).
`/cowork-plugin`	Creates a new Cowork plugin or adapts an existing one.
`/daemon`	Manage background services and routines.
`/dataviz`	Design charts and analytics pages properly.
`/debug` [issue description]	Turn on debugging for this session.
`/design` [sync|login|consent|revoke|import|export|status|<prompt>]	Grant or revoke access to design projects.

`/design-consent`	Grant access to design projects.
`/design-login`	Unlock design access for /design-sync.
`/design-revoke`	Revoke access to design projects.
`/design-sync`	Push a React design system to claude.ai/design.
`/desktop` also /app	Continue the running session in the desktop app.
`/diff`	Look at uncommitted changes and changes per turn.
`/doctor` also /checkup · [prompt-audit [<path>]]	Examine the installation and fix problems.
`/effort`	Set the thinking effort.
`/exit` also /quit	End the session.
`/explain-usage`	Have your own usage explained to you.
`/export` [filename]	Write the conversation to a file or the clipboard.
`/extra-usage`	Renamed to /usage-credits.
`/fast` [on|off]	Toggle fast mode (same model, higher speed).
`/feedback` [report]	Send feedback to Anthropic.
`/fewer-permission-prompts`	Spot frequent harmless calls and allow them for good.
`/focus` [on|off]	Show only the input, the summary and the answer.
`/fork` <directive>	Start a background agent that inherits the whole conversation.
`/goal` [<condition> | clear]	Set a goal Claude checks before it stops.
`/heapdump`	Write a memory dump (for troubleshooting).
`/help`	Show help and the available commands.
`/hooks`	Look at the hooks configured for tool events.
`/ide` [open]	Manage the connection to the development environment.
`/import` [codex|gemini|cursor] [--dry-run]	Take over the configuration of another AI tool.
`/init`	Generate a first CLAUDE.md from the existing code.
`/insights`	Build a report about your own sessions.
`/install` [options]	Install the native build.
`/install-github-app`	Set up Claude GitHub Actions for a repository.
`/install-slack-app`	Install the Claude app for Slack.
`/keybindings`	Open the keyboard shortcuts file.
`/limit-reset`	Reset your session limit now and keep working; available once a week, still counts toward the weekly limit.
`/list-agents` also /peers	List subagents and other sessions you can reach.
`/login`	Sign in to the Anthropic account or switch accounts.
`/logout`	Sign out of the Anthropic account.
`/loop` also /proactive · [interval] [prompt]	Run a task over and over at a fixed interval.
`/low-priority`	Continue after hitting the session limit at lower priority; run again to stop.

`/mcp` [reconnect (<server>|all)|enable|disable [<server>|all]]	Manage MCP servers.
`/memory`	Edit CLAUDE.md files and the memory settings.
`/memory-types`	Reference on the kinds of memory: what belongs where, and when.
`/mobile` also /ios, /android	Show the QR code for the phone app.
`/model` [model]	Set the model for Claude Code.
`/output-style` [style]	List output styles or switch to one.
`/passes`	Give away a free week of Claude Code.
`/permissions` also /allowed-tools	Manage the rules for allowed and forbidden tools.
`/plan` [open|<description>]	Turn on plan mode or look at the session's plan.
`/plugin` also /plugins, /marketplace	Manage plugins.
`/plugin-authoring`	Helps you write and debug a plugin made of function hooks.
`/powerup`	Get to know Claude Code's features in short lessons.
`/privacy-settings`	View and change the privacy settings.
`/pro-trial-expired`	Your options once the Pro trial has run out.
`/prototype`	Turns an idea into a working draft and publishes it as a page.
`/radio`	Turn on the lo-fi radio.
`/rate-limit-options`	Show the options once the usage limit is reached.
`/recap`	Produce a one-line summary of the session.
`/release-notes`	Show what is new in each version.
`/reload-plugins` [--force]	Apply pending plugin changes.
`/reload-skills`	Read in newly added or changed skills.
`/remote-control` also /rc	Drive the session from your phone or from claude.ai.
`/remote-env`	Choose the default environment for cloud agents.
`/rename` also /name · [name]	Rename the conversation.
`/restart` also /update	Restart Claude Code and keep this session.
`/resume` also /continue	Continue an earlier conversation.
`/rewind` also /checkpoint, /undo	Roll code and/or conversation back to an earlier state.
`/run`	Start the application and check a change in real operation.
`/run-skill-generator`	Turn a workflow of yours into a new skill.
`/sandbox`	Set up and inspect the sandbox.
`/schedule` also /routines	Create and manage scheduled cloud agents.
`/scroll-speed`	Set the scroll speed of the mouse wheel.
`/session` also /remote	Show the address and QR code of the cloud session.
`/setup-bedrock`	Set up Amazon Bedrock again.
`/setup-claude` also /setup-cowork	Walks you through setup: pick a role, install a matching plugin, try a skill, connect tools.
`/setup-vertex`	Set up Google Vertex AI again.

`/simplify` [<target>]	Tidy up and simplify changed code.
`/skill-doctor`	Show which loaded skills are costing context without being used.
`/skills`	List the available skills.
`/status`	Show the state: version, model, account, connections.
`/statusline`	Set up the status line.
`/stickers`	Order Claude Code stickers.
`/stop`	End this background session; log and worktree are kept.
`/subtask` <task>	Send off a subagent with full context; the result comes back.
`/tasks` also /bashes	View and manage everything running in the background.
`/team-onboarding`	Generate a guide for colleagues out of your own usage.
`/teleport` also /tp	Continue a session from claude.ai here.
`/terminal-setup`	Set up the terminal key bindings (line break).
`/theme`	Switch the colour scheme.
`/tui` [default|fullscreen]	Choose the interface style: normal or full screen.
`/ultraplan` <prompt>	Have an editable plan drafted on the web.
`/ultrareview`	Multi-stage bug hunt across the branch, in the cloud.
`/upgrade`	Move to Max for higher limits and more Opus.
`/usage` also /cost, /stats	Show the session's cost, usage and activity.
`/usage-credits`	Set up credits or ask your administrator for them.
`/verify`	Checks that a change really does what it should — in a real run.
`/voice` [hold|tap|off]	Toggle voice mode.
`/web-setup`	Connect Claude Code on the web with your GitHub account.
`/workflow-launch-exec`	Execute a workflow triggered by the server.
`/workflows`	Look through running and finished workflows.
`/workshop`	Designs something with you step by step, as a growing document.

Appendix · Present, but switched off

These names exist in the program, but are disabled by a flag in this version and are not available inside a session.

`/loops`	List, create and delete loops. present in the program, but switched off
`/pause-memory` also /memory-pause, /toggle-memory	Pause account memory and local auto-memory for this session (run again to turn both back on). present in the program, but switched off
`/version`	Show the version of this session. present in the program, but switched off
`/wellbeing` also /breaks, /break-reminder, /downtime	Set up break reminders and quiet hours. present in the program, but switched off